# Identifier-Aware Graph Retrieval for Bug Localization with Reduced Generative LLM Dependence

Author names and affiliations pending

Working preprint draft — November 2, 2026

**Abstract**

Graph-based bug localization can depend on a generative large language model (LLM) both to discover candidate code entities and to identify final modification locations. We investigate replacing the discovery stage with a bounded retrieval procedure over GraphLocator's repository graph. The procedure combines BM25, reserved slots for classes explicitly named in an issue, semantic disambiguation of those class matches, and relevance-ranked one-hop expansion. We analyze existing development experiments on 18 Jackson-core issues and 18 Django issues, without running new retrieval or model experiments. A lexical class-match variant with expansion retrieves every ground-truth file on 15/18 Jackson-core issues; the CodeXEmbed class-match variant without expansion reaches 14/18. On Django, CodeXEmbed with expansion increases complete file coverage from 8/18 for the lexical variant to 10/18, while an LLM SearchAgent reaches 15/18. Expansion preserves additional function candidates even when file coverage is unchanged. Recomputed candidate-set precision remains low: 18.0% at file level for the Jackson-core expanded lexical variant and 8.9% for the Django expanded CodeXEmbed variant. These findings support separating broad candidate discovery from final selection, but do not establish end-to-end localization performance or runtime savings. A small-LLM reranking stage is proposed as the next component; it has not been evaluated. The study identifies name collisions, candidate reachability, and granularity as constraints on reducing generative LLM dependence.

## 1 Introduction

Bug localization maps a natural-language issue to the code that requires modification. Repository graphs offer a useful search space because they expose files, classes, functions, and relationships among them. GraphLocator combines such a representation with LLM-guided search and subsequent causal reasoning [1]. Its search stage finds symptom-related code; its causal stage builds and refines a causal issue graph to investigate underlying causes.

This study starts from a narrower question: which discovery decisions need to be made by a generative LLM? An issue often contains explicit identifiers, and a repository graph already records containment and dependency information. Lexical retrieval can exploit the first signal, while graph expansion can expose related entities. A learned embedding model can disambiguate a small candidate pool without generating a sequence of search actions. These observations motivate a pipeline in which retrieval constructs a bounded context before a small LLM makes a final selection.

The objective is reduced *generative* LLM dependence, rather than the removal of all learned models. UniXcoder and CodeXEmbed remain pretrained neural components when used for semantic scoring. The completed experimental component is candidate discovery. Final prediction using a small LLM remains unfinished. Accordingly, this article reports an exploratory retrieval study and a clearly

separated design for the next stage. It does not claim a completed replacement for GraphLocator's causal reasoning, state-of-the-art localization, or measured end-to-end cost reductions.

We examine three questions. **RQ1:** How well do identifier-aware lexical and semantic variants cover ground-truth entities? **RQ2:** What coverage is lost when a ten-entity base is used without graph expansion? **RQ3:** What limits the ability of a later reranker to turn these candidates into final predictions?

The contribution is an analysis of a concrete bounded retrieval design and its development evidence: reserving slots for explicitly named classes, using representation-enriched semantic scoring to address name collisions, and retaining expansion when it contributes otherwise missing entities. The components themselves are established techniques. Their combination is evaluated as an engineering hypothesis, without claiming that this is the first hybrid retrieval method for bug localization.

## 2 Background and Related Work

**Graph-guided localization.** GraphLocator addresses symptom-to-cause and one-to-many mismatches through a repository dependency graph and a causal issue graph [1]. We reuse its graph infrastructure and entity formatting, while studying a replacement for its initial SearchAgent. Graph adjacency is used here to limit candidate eligibility; an adjacent entity is not assumed to be a cause of the reported defect. Agentless provides a related motivation for structured localization workflows rather than unrestricted agent interaction [6]. Our study concerns candidate retrieval, not patch generation or repair success.

**Lexical and semantic retrieval.** BM25 provides an established term-matching model [2]. UniXcoder learns code representations using code and cross-modal information [3]. CodeXEmbed is a family of models for multilingual and multi-task code retrieval [4]. SPLADE-Code studies learned sparse retrieval for code [5]. We use these models as existing components, not models trained by this study. Their local rankings should not be interpreted as a general comparison of model families.

**Representation and context selection.** Caumartin et al. investigate how file representations affect retrieval and post-retrieval ranking [7]. Their work motivates treating candidate text as a design variable. Our unit is a graph entity rather than a whole-file representation, and our most informative representation experiment concerns disambiguating production classes from same-named test fixtures. We do not evaluate generated role-aware summaries. A systematic comparison with the broader graph-retrieval literature remains necessary before a stronger novelty claim can be made.

## 3 Problem Formulation and Scope

Let $q$ be an issue description and $G = (V, E)$ a repository graph. A vertex has a type, identifier, source location, and available code text. An edge represents containment or a dependency exposed by the graph implementation. Candidate discovery returns $C(q) \subseteq V$ with a small entity budget. An eventual selection stage returns $P(q) \subseteq C(q)$.

Let $T_i^{(g)}$ be the ground-truth set for issue $i$ at granularity $g \in \{\text{file}, \text{module}, \text{function}\}$, and let $R_i^{(g)}$ be the corresponding entities represented by retrieved candidates. "Module" follows the existing

GraphLocator evaluator: it generally denotes a class-like entity, not necessarily a Python module. Retrieving a class credits its file and class, but does not automatically credit every contained function.

For a fixed candidate universe, a selector cannot recover an absent entity:

$$P_i^{(g)} \subseteq R_i^{(g)} \quad \Longrightarrow \quad \mathrm{Recall}(P_i^{(g)}, T_i^{(g)}) \leq \mathrm{Recall}(R_i^{(g)}, T_i^{(g)}). \tag{1}$$

This ceiling changes if the final stage is allowed to inspect additional members of a retrieved class or file. Such inspection is a further localization stage, not reranking over the original fixed set, and is outside the measured results.

# 4 Identifier-Aware Retrieval over the Repository Graph

Figure 1 distinguishes the evaluated retrieval components from the proposed final selector. The semantic class-match variant uses a ten-entity base and appends up to five neighbors; no candidate-threshold filter is required by the selected expanded design.

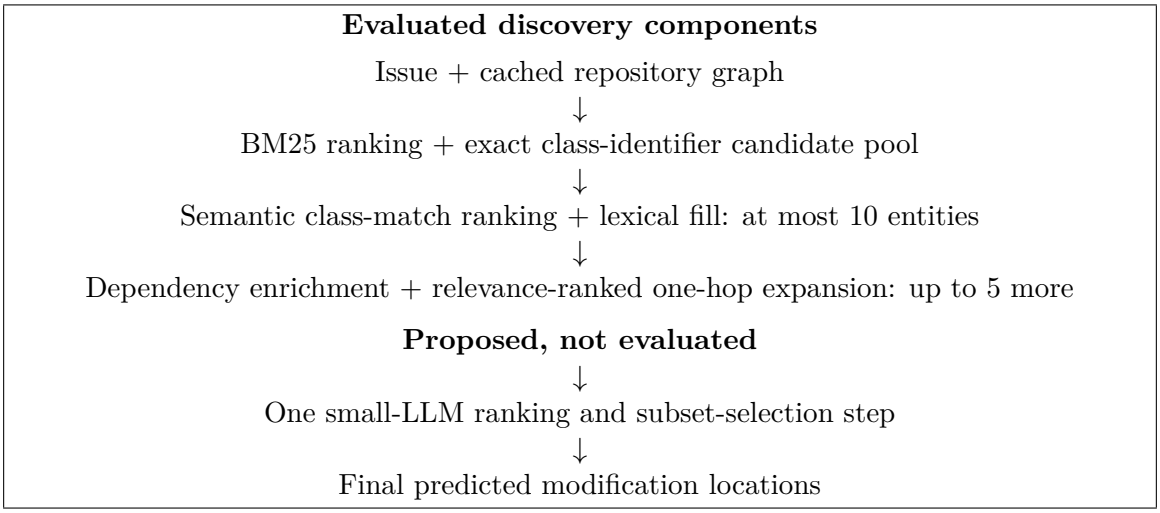



Figure 1: Intended architecture. Different discovery variants have different experimental support; the final generative selection stage has no reported results.

## 4.1 Lexical scoring and identifier extraction

Each initial vertex is represented by its name and available content. The local tokenizer lowercases identifier-like tokens; it does not split CamelCase into subwords. The implementation uses BM25 with $k_1 = 1.5$ and $b = 0.75$, then adds an identifier-match bonus:

$$s(v, q) = \mathrm{BM25}(q, \mathrm{name}(v) \| \, \mathrm{content}(v)) + 5 \mathbb{1}[\mathrm{name}(v) \in I(q)], \tag{2}$$

where $I(q)$ contains identifiers extracted from backtick-quoted and mixed-case spans, including components of dotted identifiers. Exact matching also uses a case-insensitive comparison. Only positively scored vertices enter the initial ranking. Let $S_{10}$ be its first ten vertices.

Early experiments showed why a small additive bonus was insufficient. A named class could score below incidental lexical matches by much more than five points. Increasing the ranking depth

helped, but did not consistently preserve these explicitly named entities. Reserved class-match slots therefore became a separate mechanism rather than another small score adjustment.

### 4.2 Reserved class matches and semantic disambiguation

The additional class-match pool is

$$A = \{v \in V \setminus S_{10} : \text{type}(v) = \text{CLASS},\ \text{name}(v) \in I(q)\}. \tag{3}$$

The implementation excludes the entire initial $S_{10}$ from this additional pool. It selects at most five extras, denoted $A_m$, then forms

$$B = \text{prefix}_{10-m}(S_{10}) \parallel A_m, \qquad m \leq 5. \tag{4}$$

Thus, the design does not always use exactly five lexical seeds and five semantic extras: lexical candidates occupy additional slots when fewer class extras are available. The budget is a maximum, not a guarantee of ten distinct valid outputs on every possible issue.

The lexical variant ranks $A$ by $s$. Semantic variants rank it by cosine similarity between issue and candidate embeddings. Enriched class text includes the identifier, repository-relative path, up to 300 characters of available content, and up to 30 member names when the containment traversal exposes them. The CodeXEmbed implementation uses `Salesforce/SFR-Embedding-Code-400M_R`, mean pooling with attention masking, and normalized vectors. The embedding input limit is 256 tokens; issue text is first limited to 2,000 characters. Consequently, available metadata is not necessarily fully visible to the encoder.

Class-only reservation is intentional. Early checks allowing every identifier type admitted generic method names and displaced useful candidates. This restriction is empirically motivated on the development cases; it is not a proof that class matches are always preferable or that method identifiers are unhelpful.

### 4.3 Dependency enrichment and one-hop expansion

The selected base entities trigger incremental dependency enrichment. The expanded graph supplies incoming and outgoing neighbors over the available relation types. For $N(B)$, the union of those one-hop neighbors outside $B$, the expanded candidate list is

$$C = B \parallel \text{Top5}_s(N(B) \setminus B), \qquad |C| \leq 15. \tag{5}$$

Graph structure determines which extras are eligible; lexical relevance ranks them. The score map comes from the initial graph. A vertex introduced only by enrichment receives the implementation's default score of zero if absent from that map. This is an implementation limitation, not semantic scoring of every newly discovered neighbor.

Dependency enrichment can be expensive and is distinct from BM25 arithmetic. Early skeleton graphs primarily exposed containment; richer dependencies became available through incremental analysis. Evidence based on a cached skeleton's neighbors must therefore be distinguished from evidence on an enriched graph.

# 5 Study Design and Evidence

## 5.1 Development cohorts and provenance

The analysis covers two cohorts already present in the workspace: 18 Jackson-core issues from GraphLocator's Multi-SWE-bench Java data and 18 Django issues from its LocBench data. These are repository-specific development subsets, not the complete benchmarks. Method choices were repeatedly informed by these issues. Django initially served as a transfer check, but became development data after its failures guided semantic disambiguation. Neither cohort is a held-out test set for the final design.

The principal evidence is the saved result JSON, supplemented by the experiment diary and its source tables. Django's original SearchAgent and lexical baseline combine a 17-issue batch with an earlier smoke-test issue, `16421`. We recompute set intersections from stored predictions and the supplied three-level ground truth; no repository checkout, search execution, or model inference was performed for this manuscript. The recomputed coverage categories agree with the stored labels for the directly audited configurations.

The original Django SearchAgent used local `qwen3:14b` with a five-turn search budget. It returned variable-size lists, unlike the heuristic's fixed maximum budgets. This comparison is informative but not budget-matched. Earlier Jackson-core LLM logs record outgoing search requests rather than complete returned candidates. Their identifier-name coverage is not used as an equivalent baseline for the audited heuristic sets.

One Jackson-core semantic-plus-expansion comparison in the diary consists of 11 independent UniXcoder reruns and seven values carried from the lexical expanded baseline. We identify that provenance rather than presenting it as 18 independent CodeXEmbed expanded runs. The complete Jackson-core expanded batch reported below is the lexical class-match variant; the complete CodeXEmbed expanded batch is on Django.

## 5.2 Metrics

For nonempty ground truth, we report *complete coverage* (CC) when $T_i^{(g)} \subseteq R_i^{(g)}$, *partial coverage* when their intersection is nonempty but incomplete, and *none* when it is empty. The historical evaluator calls CC "EM." We avoid that term because false-positive entities are allowed. Exact predicted-set equality is a different condition.

We additionally recompute precision, recall, and F1:

$$\mathrm{P}_i = \frac{|R_i \cap T_i|}{|R_i|}, \quad \mathrm{R}_i = \frac{|R_i \cap T_i|}{|T_i|}, \quad \mathrm{F1}_i = \frac{2\mathrm{P}_i\mathrm{R}_i}{\mathrm{P}_i + \mathrm{R}_i}. \tag{6}$$

Empty predictions receive zero precision; F1 is zero when both scores are zero. Macro averages are taken over issues with ground truth at the relevant level. Five Jackson-core issues lack function ground truth, leaving 13 eligible issues; two Django issues lack module ground truth, leaving 16. No significance or population-generalization claim is made for these small, adaptively examined cohorts.

# 6 Results

## 6.1 RQ1: Coverage and candidate precision

Table 1 summarizes the directly audited configurations. On Jackson-core, the lexical class-match variant with expansion has complete file coverage on 15/18 issues and partial coverage on the remaining three. The CodeXEmbed variant without expansion has 14 complete and four partial cases. On Django, semantic disambiguation raises complete file coverage from 8/18 to 10/18, but the LLM SearchAgent reaches 15/18. The current heuristic therefore does not replace the baseline's discovery capability across both repositories.

| Cohort | Configuration | File CC | Partial | None | Module CC | Function CC |
|---|---|---|---|---|---|---|
| Jackson | Lexical + expansion | 15/18 | 3 | 0 | 13/18 | 4/13 |
| Jackson | CodeXEmbed, no expansion | 14/18 | 4 | 0 | 12/18 | 3/13 |
| Django | LLM SearchAgent | 15/18 | 1 | 2 | 8/16 | 5/18 |
| Django | Lexical + expansion | 8/18 | 2 | 8 | 5/16 | 6/18 |
| Django | CodeXEmbed + expansion | 10/18 | 2 | 6 | 7/16 | 6/18 |
| Django | CodeXEmbed, no expansion | 10/18 | 2 | 6 | 7/16 | 3/18 |

Table 1: Search-stage coverage. CC means every ground-truth entity is present; irrelevant candidates may also be present. No row is final small-LLM prediction. Rows differ in candidate budgets and, across repositories, in execution path.

Complete coverage alone gives an overly favorable impression of the final selection problem. Table 2 shows low heuristic candidate-set precision, even for high-coverage Jackson-core retrieval. None of the 18 Jackson-core expanded lexical file sets equals its ground-truth file set exactly. The intended reranker must reject substantial irrelevant context; retrieval coverage does not establish that it can do so reliably.

| Cohort | Configuration | Precision | Recall | F1 | File CC |
|---|---|---|---|---|---|
| Jackson | Lexical + expansion | 18.0 | 90.7 | 28.7 | 83.3 |
| Jackson | CodeXEmbed, no expansion | 19.5 | 88.9 | 30.7 | 77.8 |
| Django | LLM SearchAgent | 68.3 | 86.1 | 71.7 | 83.3 |
| Django | Lexical + expansion | 7.3 | 50.0 | 12.5 | 44.4 |
| Django | CodeXEmbed + expansion | 8.9 | 61.1 | 15.3 | 55.6 |
| Django | CodeXEmbed, no expansion | 10.4 | 61.1 | 17.5 | 55.6 |

Table 2: File-level percentages recomputed from stored candidates. Precision, recall, and F1 are macro averages; CC is an issue rate. The LLM baseline generally returns fewer entities, so these are observed trade-offs, not an equal-budget model ranking.

## 6.2 Why Django required semantic disambiguation

Django issue `18906` illustrates a class-name collision. Its exact-identifier pool contained 75 class candidates, including repeated test fixtures. The real `BaseForm` ranked approximately 51st by the lexical score. A name-grouping rule improved the ordering among same-named alternatives, but did not overcome competition from other identifier groups: the representative `BaseForm` ranked seventh among nine identifiers, outside a five-extra budget.

UniXcoder with bare declaration text placed the target 35th. Enriching the text with path information moved it to second. In the recorded diagnostic, a containment traversal gap prevented the relevant `full_clean` member name from entering the representation, so that member cannot explain the observed gain. The evidence supports the usefulness of contextualized text in this case, but is not a full factorial attribution of every representation field.

On the same candidate pool, CodeXEmbed placed `BaseForm` first, whereas SPLADE-Code placed it twentieth. All three semantic models placed the easier `ErrorList` target first on issue `18905`. These are two diagnostic cases, not a broad claim that one embedding architecture dominates another. The later full Django CodeXEmbed batch recovers complete file coverage for both issues, but retrieves neither target function. Finding the correct class does not resolve function-level localization.

### 6.3 RQ2: What expansion contributes

The diary's Jackson-core ablations identify two concrete regressions when expansion is removed. Issue `729` loses `FilteringParserDelegate.java`, reducing complete file and module coverage to partial coverage; its function coverage falls from partial to none. Issue `891` retains its ground-truth file but loses the function `FilteringGeneratorDelegate.writeStartObject`. Its function coverage falls from complete to none. The CodeXEmbed no-expansion batch has the same entity sets as the UniXcoder no-expansion batch, ignoring list order across the 18 cases; changing the class-match encoder does not repair these omissions.

Four Jackson-core issues lack complete file coverage in the CodeXEmbed no-expansion batch: `1182`, `1016`, `729`, and `370`. Their missing files include `JsonParserDelegate.java`, parallel parser implementations, and `FilteringParserDelegate.java`. All 18 still contain at least one correct file. An LLM selecting only from this set cannot attain complete file coverage on those four issues.

On Django, the saved ten-entity reconstruction and the full fifteen-entity CodeXEmbed run have identical file and module coverage categories for every issue. Expansion nevertheless changes function coverage on three cases: `16421` and `18850` move from partial to complete, while `13134` moves from none to complete. Complete function coverage therefore increases from 3/18 to 6/18. Function-level macro recall increases from 28.1% to 39.3%, while precision decreases from 6.9% to 6.0%. The value of expansion depends on the required localization granularity, not just on whether it adds a new file.

Two edge-type-IDF Steiner-bridging trials did not improve the examined failure sets; the first, on seven non-complete issues, introduced one regression, and the second, on four partial issues, changed no coverage outcomes. This negative evidence favors relevance-ranked neighbors within the tested designs. It does not establish that arbitrary multi-hop graph methods are ineffective.

### 6.4 RQ3: Candidate eligibility and filtering limits

Six Django issues have no correct file in either CodeXEmbed configuration: `18435`, `19043`, `6478`, `18986`, `18941`, and `18911`. Two more, `18616` and `18854`, are partial. A recorded reachability check on cached skeletons found no neighbor in any ground-truth file for the six complete failures. For the two partial cases, the reported ground-truth neighbors were in files already represented by the base candidates. These observations concern cached skeletons, not every possible enriched graph. They indicate that rescoring an unchanged skeleton-level one-hop pool cannot solve those missing-file cases by itself.

The diary also explores discarding low-similarity candidates. On Jackson-core, score thresholds were selected using the same labeled cases on which they were reported; they are development fits rather

than held-out validation. A later Django run applies the bare-text threshold $t = 0.5700$, reducing 270 candidates to 193 overall. Complete file coverage stays at 10/18, but issue `18854` loses its only correct file and changes from partial to none. File macro precision increases from 8.9% to 12.1%, while macro recall falls from 61.1% to 58.3%. An unchanged complete-coverage count therefore conceals a real regression.

The retained design uses expansion without this threshold filter. This choice preserves candidates for the proposed selector; it is not evidence that retaining all fifteen always improves final accuracy.

# 7 Reducing LLM Dependence: Proposed Final Selection

The intended next stage is a small generative LLM that receives the issue and a bounded representation of the expanded candidates. It ranks the supplied entities and selects those likely to require modification. Candidate IDs should ground every selection, with multiple selections and explicit abstention permitted. A fixed output schema would separate malformed responses from valid empty predictions. These are proposed requirements; no model, prompt, snippet budget, or output cutoff is validated by the reported experiments.

This architecture allocates generative reasoning to final selection rather than to repeated discovery actions. The discovery stage makes no generative search calls, but uses neural embedding inference in semantic variants. A future single-call selector would reduce the number of generative interaction points by design. It would not automatically reduce wall-clock time, energy, monetary cost, or error rate: graph construction, dependency enrichment, embedding inference, and longer reranking prompts must also be counted.

An evaluation should freeze the retrieval configuration before collecting new issues, use identical candidate archives across reranker comparisons, and report retrieval ceilings alongside final precision and recall. Comparisons should include lexical-only selection, embedding reranking, the original SearchAgent, and the original full GraphLocator pipeline when evaluating final localization. Call count, input/output tokens, inference time, graph-analysis time, failure rate, and repeated-run variability should be recorded separately. No anticipated values are substituted for these missing measurements.

# 8 Discussion and Threats to Validity

**Coverage is a prerequisite, not a final result.** Jackson-core's 15/18 complete-file result is encouraging for candidate discovery, but its 18.0% macro precision exposes the unresolved selection burden. On Django, the heuristic remains behind the LLM SearchAgent in file recall and precision. A small reranker cannot compensate for all discovery failures under the fixed-set constraint in Eq. 1. Whether it can preserve coverage while removing irrelevant candidates is an open empirical question.

**Adaptation and limited external validity.** The method was repeatedly adjusted on 36 issues from two repositories. The datasets differ in language, graph representation, issue wording, and naming practices; observed differences cannot be attributed solely to Java versus Python. The exploratory ablations isolate some engineering decisions but do not replace a frozen, independently evaluated protocol. No population-level confidence or comprehensive novelty claim follows from this evidence.

**Implementation and provenance.** Early basename-only comparisons are not interchangeable with later full-path, three-level evaluation. Some semantic expanded results were carried forward rather than independently rerun. Member extraction gaps, input truncation, and zero-score enriched vertices can affect candidate ordering. Tied scores can also depend on graph iteration order. These details constrain reproducibility and the interpretation of purportedly deterministic search.

**Timing is not a controlled efficiency experiment.** Stored mean search times are about 185.0 seconds for the Jackson-core CodeXEmbed no-expansion batch, 93.8 seconds for Django's offline ten-entity reconstruction, and 3,240.6 seconds for Django's live expanded batch. The Django reconstruction skips dependency enrichment and shares a loaded encoder, while the live batch performs additional analysis and loading work. Hardware and load were not controlled in these comparisons. The values document the runs; they do not establish a causal speedup from removing five neighbors or a fair speed ranking between repositories.

**Causal-agent failures and unfinished prediction.** Earlier full-pipeline trials were affected by strict Mermaid parsing, prompt size, and connection timeouts. Some failures produced no usable final output despite relevant search candidates. These observations motivated stage-wise measurement, but do not demonstrate that a small final reranker avoids those problems or improves accuracy. A directory labeled "full pipeline check" in the later CodeXEmbed experiments still evaluates search and similarity filtering, not the proposed generative final prediction.

**Ground truth and repair.** Patch-derived entity labels specify the locations credited by the benchmark. An implementation helper may be related to the issue without being an annotated edit target. Conversely, a correct file candidate can omit the precise modified method. We report exact string matches to the supplied labels and do not claim causal correctness, successful patches, or functional repair from retrieval alone.

## 9 Conclusion

The recorded experiments show that a bounded heuristic can discover useful code context without generative search calls, but its reliability varies substantially by repository and granularity. Reserved class-identifier matches protect explicit issue evidence; semantic context helps distinguish production classes from same-named fixtures; and one-hop expansion preserves additional file or function coverage. The strongest observed expanded results cover every ground-truth file on 15/18 Jackson-core issues for the lexical class-match variant and 10/18 Django issues for the CodeXEmbed variant. These are development-set retrieval results with substantial irrelevant context. Final prediction with a small LLM remains to be evaluated, together with controlled efficiency and held-out generalization.

## References

[1] W. Liu, C. Peng, P. Gao, A. Liu, W. Zhang, H. Zhao, and Z. Jin. GraphLocator: Graph-Guided Causal Reasoning for Issue Localization. *Proceedings of the ACM on Software Engineering*, 3(FSE), Article FSE051, 2026. `https://doi.org/10.1145/3797079`.

[2] S. Robertson and H. Zaragoza. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. https://doi.org/10.1561/1500000019.

[3] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin. UniXcoder: Unified Cross-Modal Pre-training for Code Representation. In *Proceedings of ACL*, pp. 7212–7225, 2022. https://aclanthology.org/2022.acl-long.499/.

[4] Y. Liu, R. Meng, S. Joty, S. Savarese, C. Xiong, Y. Zhou, and S. Yavuz. CodeXEmbed: A Generalist Embedding Model Family for Multilingual and Multi-task Code Retrieval. *COLM*, 2025. https://openreview.net/forum?id=z3lG70Azbg.

[5] S. Lupart, M. Louis, T. Formal, H. Déjean, and S. Clinchant. On the Challenges and Opportunities of Learned Sparse Retrieval for Code. arXiv:2603.22008, 2026. https://arxiv.org/abs/2603.22008.

[6] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang. Agentless: Demystifying LLM-based Software Engineering Agents. arXiv:2407.01489, 2024. https://arxiv.org/abs/2407.01489.

[7] G. Caumartin, T.-H. Chen, and D. E. Costa. Retrieval-Oriented Code Representations in Agentic Bug Localization. arXiv:2607.11046, 2026. https://arxiv.org/abs/2607.11046.

# A Evidence Inventory and Unresolved Items

The companion `evidence_summary.json` records recomputed aggregates; `per_issue_evidence.csv` records issue-level intersections and missing entities. They are arithmetic audits of saved results, not new retrieval runs. The following identifiers map manuscript claims to the existing experiment records under `GraphLocator-HeuristicSearch/results/`:

**E1: Jackson-core expanded lexical baseline.**
Experiment 10, `batch_jackson_18_3level_results.json`.

**E2: Jackson-core CodeXEmbed without expansion.**
Experiment 12, subdirectory 5.1, `batch_jackson_18_codexembed_results.json`.

**E3: Django LLM and lexical baselines.**
Experiment 10, `batch_django_17_3level_results.json`, plus `smoke_test_django_3level.log` for issue 16421.

**E4: Django CodeXEmbed with expansion and threshold ablation.**
Experiment 12, subdirectory 6, `batch_django_18_full_pipeline_results.json`; use `found_pre` for unfiltered search and `found_post` for thresholding.

**E5: Django ten-entity reconstruction and reachability diagnostic.**
Experiment 12, subdirectory 6.1, `rerun_django_top10_results.json` and `reachability_check.log`.

**E6: Development history and diagnostic comparisons.**
`results_comparison_tables.pdf` and its LaTeX source; the class-match diagnostics in experiments 11 and 12; the saved Steiner trials in experiments 6 and 9. These do not all share the same evaluation protocol or sample size.

Author identities and affiliations, an artifact release location, a frozen evaluation split, and the final reranker specification remain to be supplied. The present text is a working manuscript, not a claim that those steps have been completed. The KGCompass directory is reference material and supplies none of the experimental results attributed to this study.